SQL (Oct 28th)

The table or structure of data is stored in database

we use SQL workbench to run all the nessesary work inside that we have SQL server which holds the nessary information in a segrated manner inside that we have database which have all the rows & collums.

Sql Comands

ddl - data definition language (defining the structure of data)

dml - data manipulative language (adding, removing, updating manipulating the data we have)

dql - data query language (questing the data "select" statement)

dcl - data control language (defining the data and controlling the data)

tcl - transation control language (having multiple transactions in one)

Creating the database

- CREATE DATABASE

CREATE TABLE test table
(table name)
id
name

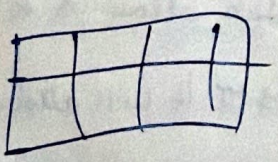
ddl, dml, dql - SELECT NULL
ALTER NOT NULL
CREATE UNIQUE
INSERT DROP
DELETE

DENORMILISATION - combining the data and putting into one

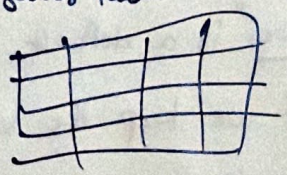
NORMILISATION - organizing data in relational database. to reduce duplication data.

Can only create one primary key
Primary key - uniquely identifing the data in table
foreign key - create as many as we need

eg - Primary key
customer table



foreign key
sales table.



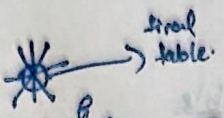
Reference table
Refered table
Referring table
Parent table
Child table

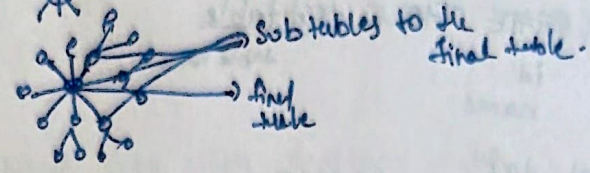
Identifiting the sales by mapping the data from customer

we can use one column as a primary key or we can use multiple.
Combined column ~~name which~~ in one column (or single column) as a primary key

CASCADE -
RESTRICT -

Schema.

Star schema - 

Snowflake schema - 

Check + default

Constraints for check - Checking the condition weather to allow it or not

default - by default itself it is storing the data we entered.

Truncate - delete the data in all the rows without deleting schema

delete + drop - delete all the data including the schema

delete the data without deleting the schema in a row.

"Safe updates" is a default is '1' which allow us to delete so that if we keep the value as '0' it will allows us to even delete that

* we can keep the safe update active when it the table we created is active.
* why => helps prevent accidentally updating or deleting large number of records in a table

Oct 29th

SELECT - Selecting the data

DISTINCT - remove the duplicates and display the data

LIMIT - display till the limit we entered

Sorting - we can sort data by ascending or decending
(default it do ascending)
"asc" "dsc"
Command - ORDER BY

Conditions
"AND" "OR" - This or that
↳ this and the other

"NOT" - [!=] symbol
↳ which is not equals will displays

multiple conditions for not is "NOT IN (data)"

LIKE
wild card filters - when we have to know the specific character wheather its a first name or last name or any columns
we use LIKE (%) to find the data.
('_')
a'%a - starting with a
'%ay% - contains a
a%a - ending with a
we only know the 2 or 5 character is then we use '_'
'_a___5%'
1 2 3 4 5 6

NULL -
between - when we need to find the data from a certain date to date or from one to one (and)

GROUP BY - to group rows that have the same values in one or more columns. used with (COUNT(), SUM(), AVG(), MAX() & MIN())
HAVING - filter the data after aggregation
WHERE - it is done in a row level
(we can use "where" wheather to segrate the data in the table level itself)

Aliasing - Syntax: Columnname As Somealiasname

* To give a temporary name to a column or table. It makes SQL queries easier to read understand & write

Initially after Aliasing in the code and when we are using the original column name it will allow us to do in sql but in further like in MySQL it won't allow us to use the aliasing name we need to use the original column name

Aliasing

Order of execution in SQL

from (table) → join → where → group by → having → select → order by → limit

Oct 30th

Cardinality relationships

1 to 1 relationship (1:1) - one record of table to one record

1 to many relationship (1:many) - eg one user can place many orders

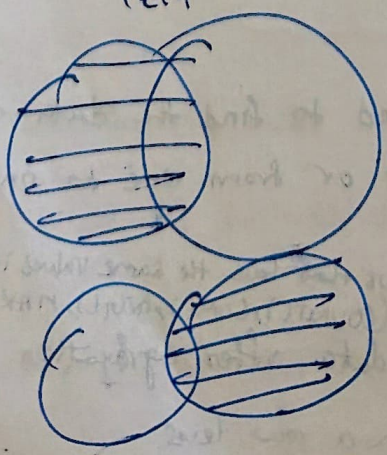
many to 1 (many:1) - eg many orders have a referece have single order

many to many (many:many) - many users are getting refered by many users

Bridge table - we use it to joined with itself (self join)

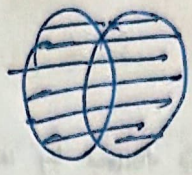
JOINS

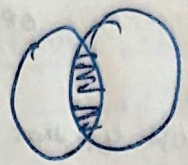
Left join

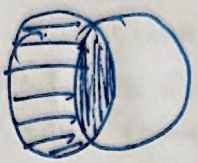


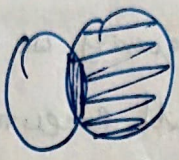
fetches the data from left table and matched data from right table

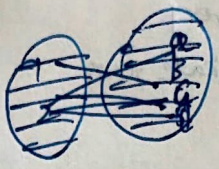
Right join

Full Outerjoin -  - display table A to B and matched data

INNER Join -  - display matched data from table A to B

LEFT JOIN Excluding INNER JOIN  - display table A without the matched data from table B

RIGHT JOIN Excluding INNER JOIN  - display table B without the matched data from table A

CROSS JOIN  - display the combination from table A to B eg- 1.a 1.c 2.b (give every combination from both tables)

FULL OUTER JOIN excluding INNER JOIN 

Sql doesn't allow to do full outerjoin we use all record from A and B and matched from A to B (we union) & union all

Both are used when the results of one SQL query to help execute another query

Subqueries - A query written inside another query

derived tables - A subquery written inside the FROM clause that acts like a temp table

limitations of subquery - we use limit in the select statement to check the limit

Correlated query - one main outerquery with an inside query

(drawback) - we cant use the subqueries multiple times (no reusability)

Nov 3rd

CTE (common Table Expression) - always follow by INSERT, UPDATE OR DELETE

Separate data from the subquery

CTE is a temporary named result set that you create using the WITH keyword

Syntax

WITH CTE Name AS (

Sub query

)

when you use CTE in multiple subqueries we don't use WITH "Code reusability" with the query level execution itself

- Separation of subquery from main query

RECURSIVE CTE - Removes duplicate rows

TEMPORARY TABLE :-

to store data temporarily while you are working with SQL

Unlike CTE, which exists only for a single SQL

⇒ If we need to reuse the data throughout the query we use temporary table.

Nov 4th

VIEWS - They are virtual tables created using SQL query

- Helps with simplifying, data abstraction, security limit

- After creating or replacing the view and doing some select query until and unless your calling the view the data will not be created in the view table.

- views are the advanced way of sharing the data in global level

- It doesnot delet the table/schema until unless we drop the views.

- while using views the input isn't be hardcoded for tha we use dynamic coding.

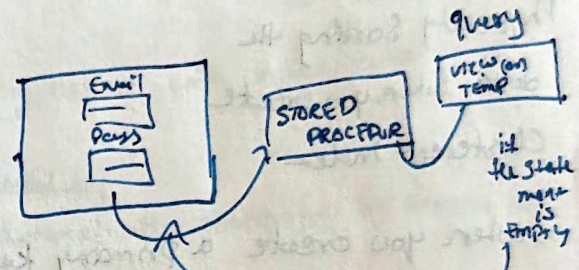
STORED PROCEDURE - helps the code to write dynamically. (Instead of writing the same SQL query multiple times, we can create a stored procedu once and call it whenever required)

- Input Parameter
- output Parameter

- can put views into stored procedure

Syntax - IN (INput Parameter help in taking the input dynamically and return the output)

```
CREATE PROCEDURE _____ (IN Parameters)
BEGIN
...
END;
```



Syntax - OUT

```
CREATE PROCEDURE _____ (IN Parameter,
                        OUT )
BEGIN
...
END;
```

- Instead of checking for every statement for the OUT (Parameter) we use IN with some statement so that it will call the function and do the necessary functions.
- INOUT - sends a value in and return an updated value

DYNAMIC SQL Procedure -

~~Instead of writing the code dynamically and the coding is written for it to give the query for itself~~

- we are making the sql to write the queries for us

Syntax - create temp table

create stored procedure

- ① we create the commands to allocate if the the tables we need
- ② insert it into the temp table

NOV 5th

INDEXING

- Instead of searching every thing for a particular file or something we find it in index and then directly search in their
- create the index in the query to optimise the search.

typeg.

clustered index

nonclustered index

↓ Adv & disadv

Physically sorting the data when you create Clustered index

→ when you create a primary key to sort the data the cluster index is created (by default it is called clustered index)

→ one cluster index is created since their is only one primary key

→ "EXPLAIN" - It will explain how the query is executed
keyword

nonclustg

→ will be created on non primary key column to help us to find the specific data

→ nonclustere is not gonna sort the data physically but keys are seperately maintained they are going to act as reference to fetch the data

→ By creating an index for the non primary key it will help us to find the specific data.

Define (taking the input form the business requirement)

↓

Design (design the query based on requirement either we need to use Primary, foreign key or clustered index, non clustered)

↓

develop (we just take the input from design)

↓

deploy

---

natural

~~Surrogate~~ key - By nature when they are created

Eg- SSN number,

- when they are created by nature with unique data is a natural key
- ~~not all primary key must be natural key~~

Surrogate key - the system is automatically generated by Primary key

- Surrogate key can be Primary key
- Every Primary key can not be Surrogate key

Eg- Employee id is a Primary key

Once the employee left it is removed.

"Partion by" - on the GROUP BY we can further segrate the data we use 'Partion By'

window function

ROW_NUMBER - assigns a unique sequential number to each row, starting from 1

RANK () [assign the same rank to a rows with equal values
Duplicate values receive same rank

DENSE_RANK - also assigns the same rank to duplicate values, it does not skip rank number

LAG () - return a value from a previous row.

LEAD () - returns a value from a following row.